CryptX — Complete Project Handbook

Files, contracts, Sepolia deployment, artwork, operations, and questions answered

18 October 2026

Read this first

Part I — What is in the project

1. Folder layout and why each folder exists
2. Root files, individually
3. Solidity source files, individually
4. Deployment, scripts and test files, individually
5. Frontend configuration and source files, individually
6. Every source artwork and metadata file
7. Technology and which component does what
8. How the final contracts were created and deployed

Part II — Contract functions and complete behavior

1. Every explicit Solidity function: access controller and Sketches
2. Every explicit Solidity function: Hidden Canvas and artwork
3. Every explicit Solidity function: marketplace and licensing
4. The separate experiment contract, function by function
5. Why the reveal has two buttons, and how recovery works
6. Frontend and file map
7. Why no private key, custom RPC URL or `.env` file is needed
8. CryptX: the finished dark-mode website redesign
9. Remaining limits

Part III — Run and operate CryptX yourself

1. Your normal start procedure on this Mac
2. Full first-time procedure after a Git clone or on a new Mac
3. How to stop the currently running site
4. Everyday website actions and what they cost
5. Recheck the project after changes

Part IV — Questions and answers

Final reference

Read this first

CryptX is a local website connected to real Ethereum Sepolia contracts. Its source code and images are on your computer or in your Git repository. The NFTs, their owners, sales, licence receipts, and public reveal state are on the Sepolia blockchain. Copying or reinstalling the website does not reset blockchain state, and opening the site does not create a new contract.

The current website is `http://127.0.0.1:5173/`. It uses React for the interface, Vite for local serving/builds, ethers for Ethereum calls, MetaMask for user signatures, Solidity/Hardhat for contracts, OpenZeppelin for ERC-721 and security helpers, and Zama FHE for encrypted seeds and delegated decryption. There is **no project backend database** and no private deployment key in the repository.

The **four deployed application contracts** are Sketches `0xE98bEd0943A1fc22296E1bBfafca30660a3b6508`, Hidden Canvas `0x8a2d41319C8bF12Cc86a10C2499d195a9dF47a84`, Marketplace `0xc6209DBeF278159DAeb383414C34266D65ec1252`, and Artwork Licensing `0x8a465728f97EEc5062Ed57E46B5659Cfbd7d9E13`. They are recorded in `deployments/sepolia-marketplace.json`, together with creation/link transaction hashes. The chain ID is `11155111`.

How to use this handbook: Part I is the file-and-folder map and deployment construction. Part II explains contract functions and the completed workflow. Part III gives exact commands for running and stopping the project yourself, plus questions and answers. Paths are relative to the project root unless stated otherwise. Third-party dependency folders are explained as generated groups rather than listing thousands of package-internal files.

Part I — What is in the project

1. Folder layout and why each folder exists

Folder	Purpose	Keep when sharing source?
`contracts/`	Solidity application contracts, the earlier feasibility contract, interfaces and test helpers.	Yes
`deployments/`	Public JSON manifest for the existing Sepolia deployment. It is how a new browser finds the same contracts.	**Yes**
`docs/`	The project walkthrough and new-device setup guide.	Yes
`frontend/`	The one-page CryptX website, artwork/metadata assets, browser tests, its own package lock and Vite settings.	Yes
`scripts/`	Reproducible artwork generation, public manifest verification, and frontend asset/ABI synchronization.	Yes
`test/`	Hardhat Solidity behavior and exact renderer parity tests.	Yes
`node_modules/`, `frontend/node_modules/`	Downloaded npm dependencies. `npm ci` recreates them from lockfiles.	No
`artifacts/`, `cache/`, `types/`, `fhevmTemp/`	Hardhat compilation and generated TypeScript/FHE outputs. `npm run compile` recreates them.	No
`frontend/dist/`, `frontend/test-results/`, `frontend/public/vendor/`	Built site, browser-test output, and copied Zama browser assets. Build/test/sync recreate them.	No

The last three groups are ignored by `.gitignore`. The source, public manifest, lockfiles and original art are the transferable project.

2. Root files, individually

File	What it does
`.nvmrc`	Pins Node `22.23.3` for `nvm use`.
`.gitignore`	Prevents committing dependencies, generated output, logs and `.env` secrets.
`package.json`	Root Hardhat project: dependencies, `compile`, Solidity `test`, and TypeScript `typecheck` scripts.
`package-lock.json`	Exact root npm dependency resolution for reproducible `npm ci`.

File	What it does
`hardhat.config.ts`	Solidity `0.8.27`, optimizer runs `800`, Cancun EVM, IR compilation for the renderer/Hidden Canvas, local test chain 31337, Sepolia chain 11155111, and optional CLI RPC/account environment values.
`tsconfig.json`	TypeScript settings for root Hardhat scripts/tests.
`README.md`	Short project description, addresses, basic start commands and limitations.
`LICENSE`	CryptX MIT licence text.
`ZAMA_TEMPLATE_LICENSE`	Retained BSD 3-Clause Clear notice for adapted Zama template material.
`CryptX_Complete_Project_Guide.pdf`	This handbook; a readable artifact, not a runtime dependency.

3. Solidity source files, individually

File	Responsibility
`contracts/SketchesNFT.sol`	ERC-721 with ten constructor-minted Sketches; records original SVG hashes and returns local metadata URLs.
`contracts/HiddenCanvasNFT.sol`	ERC-721 with paid, capped minting, one encrypted seed/controller per token, delayed preview, transfer revocation and two-stage public reveal.
`contracts/NFTMarketplace.sol`	Non-escrow fixed-price listing, cancel, buy, listing invalidation and seller pull-withdrawal.
`contracts/ArtworkLicenseRegistry.sol`	Paid, non-transferable licence receipts tied to an artwork content hash and fixed terms; creator licence proceeds.
`contracts/TokenAccessController.sol`	One controller per encrypted token; only its creating collection can activate or revoke user-decryption delegation.
`contracts/CanvasArt.sol`	Deterministic SVG/trait renderer from a `uint64` seed, kept byte-for-byte in parity with the browser renderer.
`contracts/SeedFeasibility.sol`	Earlier standalone privacy experiment retained as source/history; **not** an ERC-721 or part of the current website.
`contracts/interfaces/IProjectCollection.sol`	Common collection and marketplace call interfaces; it does not deploy a contract.
`contracts/test/CanvasArtHarness.sol`	Test-only wrapper exposing internal SVG rendering for exact comparison.
`contracts/test/AdversarialActors.sol`	Test-only rejecting-ETH seller and reentering buyer used to check failure handling.

OpenZeppelin supplies standard ERC-721 functions; the project overrides/adds the business rules described in Part II. `contracts/test/` files are local test helpers, not Sepolia application contracts.

4. Deployment, scripts and test files, individually

File	Responsibility
`deployments/sepolia-marketplace.json`	Public addresses, chain ID, creator, prices, metadata base URL, six deployment/link receipts and verified block. Keep this in Git.
`scripts/generate-sketches.mjs`	Rebuilds ten original SVGs, ten metadata JSON files, catalog entries and content hashes. Do **not** regenerate/change these casually after deployment because the on-chain hashes are fixed.
`scripts/import-market-manifest.mjs`	Checks an externally downloaded *public* marketplace manifest against Sepolia transaction data, compiled creation bytecode, links, fees and SVG hashes before saving it. Not needed for ordinary startup.
`scripts/sync-market-frontend.mjs`	Copies compiled contract ABI/bytecode, the Sketches catalog and the verified deployment manifest into `frontend/src/` before dev/build.
`scripts/sync-zama-assets.mjs`	Copies the pinned Zama SDK browser/WASM bundle from installed packages into `frontend/public/vendor/zama/`.
`test/CanvasArt.ts`	Exact Solidity-versus-browser renderer parity for selected seed boundaries and motions.
`test/MarketplaceAndLicensing.ts`	Local contract integration checks for supply, prices, listing versions, transfer, reveal, licences, withdrawals and receiver/reentrancy cases.
`test/SeedFeasibility.ts`	Local mock tests for the earlier privacy experiment, including access revocation, epochs and proof flow.

5. Frontend configuration and source files, individually

File	Responsibility
`frontend/index.html`	The **only** website HTML entry point. Bare `127.0.0.1:5173/` loads CryptX.
`frontend/package.json`	Frontend React/Vite/ethers/Zama dependencies and `sync`, `dev`, `build`, `test` scripts.
`frontend/package-lock.json`	Exact frontend npm dependency resolution.
`frontend/vite.config.ts`	Builds `index.html`, serves only `127.0.0.1`, requires port 5173, and sets COOP/COEP headers needed by browser cryptography.
`frontend/tsconfig.json`	Frontend TypeScript settings.
`frontend/playwright.config.ts`	Browser-test server URL, desktop viewport and test directory.
`frontend/src/market/main.tsx`	React UI: navigation, search, filters, wallet connection, NFT detail/actions, private preview, reveal, buying, listing, licences and balances.

File	Responsibility
`frontend/src/market/chain.ts`	Contract ABIs, addresses/manifest validation, public RPC and MetaMask providers, transaction confirmation and six-step deployment wizard.
`frontend/src/market/model.ts`	Reads Sepolia state and turns it into Sketches/Hidden Canvas cards, valid listings, receipts and balances.
`frontend/src/market/style.css`	Dark responsive marketplace design, layout and accessibility states.
`frontend/src/market/catalog.json`	Synced copy of the original Sketches catalog. Generated by the sync script.
`frontend/src/lib/art.ts`	Browser-side bigint-safe deterministic trait and SVG renderer; matches `CanvasArt.sol`.
`frontend/src/lib/zama.ts`	Loads local Zama WASM, creates fresh Sepolia SDK instances, requests delegated private decryption and classifies relayer failures.
`frontend/src/abi/SketchesNFT.json`	Compiled Sketches ABI and deployment bytecode for browser calls/wizard.
`frontend/src/abi/HiddenCanvasNFT.json`	Compiled Hidden Canvas ABI and bytecode.
`frontend/src/abi/NFTMarketplace.json`	Compiled marketplace ABI and bytecode.
`frontend/src/abi/ArtworkLicenseRegistry.json`	Compiled licence registry ABI and bytecode.
`frontend/src/abi/sepolia-marketplace.json`	Synced copy of the public deployed-address manifest used on first browser load.
`frontend/test/art.spec.ts`	Browser-side renderer, ten asset hashes, metadata routes and sealed placeholder checks.
`frontend/test/market.spec.ts`	Homepage setup, wrong manifest, mobile viewport and wrong-chain wallet UI checks.

The `frontend/src/abi/*.json` outputs can be regenerated from the Solidity artifacts and public manifest, but the root `deployments/sepolia-marketplace.json` and source files must be kept.

6. Every source artwork and metadata file

File	What it represents
`frontend/public/cryptx-mark.svg`	Original CryptX C/X logo and browser favicon.
`frontend/public/art/sealed.svg`	Generic public placeholder displayed before Hidden Canvas proof completion.
`frontend/public/art/sketches/catalog.json`	Names, descriptions, local SVG file names and content hashes for all ten Sketches.
`frontend/public/art/sketches/01.svg`	Sketches NFT #1 original vector image.
`frontend/public/art/sketches/02.svg`	Sketches NFT #2 original vector image.
`frontend/public/art/sketches/03.svg`	Sketches NFT #3 original vector image.

File	What it represents
`frontend/public/art/sketches/04.svg`	Sketches NFT #4 original vector image.
`frontend/public/art/sketches/05.svg`	Sketches NFT #5 original vector image.
`frontend/public/art/sketches/06.svg`	Sketches NFT #6 original vector image.
`frontend/public/art/sketches/07.svg`	Sketches NFT #7 original vector image.
`frontend/public/art/sketches/08.svg`	Sketches NFT #8 original vector image.
`frontend/public/art/sketches/09.svg`	Sketches NFT #9 original vector image.
`frontend/public/art/sketches/10.svg`	Sketches NFT #10 original vector image.
`frontend/public/metadata/sketches/1.json`	Sketches #1 token metadata; points to its local SVG.
`frontend/public/metadata/sketches/2.json`	Sketches #2 token metadata; points to its local SVG.
`frontend/public/metadata/sketches/3.json`	Sketches #3 token metadata; points to its local SVG.
`frontend/public/metadata/sketches/4.json`	Sketches #4 token metadata; points to its local SVG.
`frontend/public/metadata/sketches/5.json`	Sketches #5 token metadata; points to its local SVG.
`frontend/public/metadata/sketches/6.json`	Sketches #6 token metadata; points to its local SVG.
`frontend/public/metadata/sketches/7.json`	Sketches #7 token metadata; points to its local SVG.
`frontend/public/metadata/sketches/8.json`	Sketches #8 token metadata; points to its local SVG.
`frontend/public/metadata/sketches/9.json`	Sketches #9 token metadata; points to its local SVG.
`frontend/public/metadata/sketches/10.json`	Sketches #10 token metadata; points to its local SVG.

These exact SVG bytes matter: their hashes were passed into the Sketches constructor. Published Hidden Canvas paintings are **not** saved image files here; the verified `uint64` seed is rendered into SVG by Solidity and returned in on-chain base64 `tokenURI` metadata. The frontend uses the matching TypeScript renderer for private preview.

7. Technology and which component does what

Technology	Pinned role in this project
Node.js `22.23.3` / npm 10	Runs compilation, tests, scripts and local Vite development server.
Solidity `0.8.27` / Hardhat `2.28.6`	Builds/tests smart contracts; Hardhat local chain is separate from Sepolia.
OpenZeppelin Contracts `5.6.1`	Audited-library building blocks such as ERC-721 and ReentrancyGuard; this does not make this project audited.
Zama FHE Solidity `0.11.1` / browser relayer SDK `0.4.1`	Encrypted random seed, handle permissions, delegated owner preview, public decrypt proof.
ethers `6.16.0`	Reads contracts through JSON-RPC and wraps the MetaMask wallet provider for signing.

Technology	Pinned role in this project
React 19.2.0 / TypeScript 5.9.3	Typed interactive browser UI.
Vite 7.3.1	Local site at port 5173, static asset serving and production bundle.
Playwright 1.57.0	Automated browser checks without a real signing wallet.
MetaMask	Holds account keys, requests Sepolia transaction approvals and signs typed private-preview messages.
PublicNode Sepolia RPC	Public blockchain read endpoint: `https://ethereum-sepolia-rpc.publicnode.com`. No project-specific API key.

The website reads live chain state; it has no accounts table or NFT database. A **public RPC URL** is a network endpoint, not a wallet private key. Sepolia test ETH pays transaction gas even when the RPC is free. `hardhat.config.ts` can optionally read `SEPOLIA_RPC_URL` and `DEPLOYER_PRIVATE_KEY` from process environment for future CLI operations, but the current MetaMask workflow does not need either. There is no automatically loaded root `.env` file.

8. How the final contracts were created and deployed

The work began with `SeedFeasibility.sol`, a small real-Sepolia experiment that proved the intended encrypted-seed access pattern before an ERC-721 was built. That experiment is historical source only; the current website uses the later NFT contracts. The final contracts were written in Solidity, compiled with Hardhat, tested locally, and then deployed through the site's MetaMask deployment wizard using a faucet-funded Sepolia creator account. The wallet signed **six ordered Sepolia transactions**; no project file held its private key.

Order	Signed action	Why it happens in this order	Public transaction hash
1	Deploy Sketches	Constructor safely mints IDs 1–10 to creator and stores original art hashes.	`0xa4390be8145720dd316b92549a457f59ec5b658f2f1af2d3e7d7ee8b157fd048`
2	Deploy Hidden Canvas	Constructor fixes creator, `0.001` ETH mint price and the mandatory 96-block Sepolia delay.	`0x27b653c9aa7b7489c2dda9eccc1ebb016fd2968c76fd60b11ed05217fcb74a83`
3	Deploy Marketplace	Its constructor needs both deployed collection addresses.	`0x9f02e1d2772a40b2fcc595697a6e42bee83c1c37748277ee135c6e457d636a9d`
4	Deploy Licensing	Its constructor needs both collection addresses and the `0.0001` ETH licence fee.	`0x68200ec91dbae2221e3263abf0b4b932ff027b7d90febd55a86235751ddf53d6`
5	Link Sketches	Creator authorizes the marketplace once in Sketches, enabling transfer-time listing invalidation.	`0x6af03c17af56f7b0ab7e42a2c931aa05d687d1ad004b408853e15fce2537afac`

Order	Signed action	Why it happens in this order	Public transaction hash
6	Link Hidden Canvas	Creator authorizes the marketplace once in Hidden Canvas.	`0xf9999c91b75012d1bf6783948d1c50b31c5821acc7cea3e0f755ac25c5cdeafa`

The constructor inputs were: `SketchesNFT(creator, metadataBaseURI, bytes32[10] originalArtHashes)`; `HiddenCanvasNFT(creator, 0.001 ETH, 96 blocks)`; `NFTMarketplace(sketchesAddress, hiddenCanvasAddress)`; and `ArtworkLicenseRegistry(sketchesAddress, hiddenCanvasAddress, 0.0001 ETH)`. The creator was the connected MetaMask deployer. The addresses, constructor settings and successful receipts are recorded in the manifest; the importer also checked them against the deployed code/state. You should **not** rerun this deployment wizard just to install or view the project on another computer.

The internal manifest `kind` is still `canvas-market`, and some already-deployed token descriptions use that original identifier. Renaming the website to **CryptX** did not and cannot rename immutable deployed code. Changing those on-chain strings requires a new deployment.

Part II — Contract functions and complete behavior

1. Every explicit Solidity function: access controller and Sketches

`TokenAccessController.sol` is deployed once **per Hidden Canvas token**, giving each controller access to only that token's ciphertext. The experiment uses the same controller design.

Function or modifier	What it does and who can call it
`constructor()`	Records the deploying collection contract as the immutable `collection`. A wallet cannot later appoint itself as controller owner.
`onlyCollection`	Modifier that rejects calls unless the calling address is that original collection contract.
`activate(holder)`	Only the collection calls this after its owner/maturity checks. It creates a revocable Zama user-decryption delegation for that holder in the collection's context; it rejects a second active holder. This is **not** permanent `FHE.allow` permission to the wallet.
`revoke()`	Only the collection calls this. It removes the active holder's delegation and clears `activeHolder`; if nobody is active it is a no-op. An ACL failure propagates, so a transfer that requires revocation reverts atomically.

`SketchesNFT.sol` is the ordinary collection:

Function	What it does
`constructor(creator, metadataBaseURI, assetHashes)`	Rejects invalid setup, stores creator and ten exact artwork hashes, and safely mints token IDs 1–10 to the creator. There is no later mint function.
`setMarketplace(candidate)`	Creator-only, one-time link to a deployed marketplace that confirms it allows this collection. This permits trusted listing invalidation on transfers.
`totalSupply()`	Always returns 10.
`mintingClosed()`	Always returns `true`; the collection cannot be publicly minted again.
`artworkHash(tokenId)`	Returns the stored SHA-256 hash of the original SVG bytes for an existing token. The licensing registry uses this as the content reference.
`canList(tokenId)`	Returns `true` for an existing Sketches NFT; the marketplace still checks actual ownership and approval.
`tokenURI(tokenId)`	Returns the configured metadata URL plus `<tokenId>.json`; in this deployment it points to the local Vite server.

Function	What it does
`_update(to, tokenId, auth)`	Internal OpenZeppelin ERC-721 transfer/mint hook. On any post-mint transfer, including a self-transfer, it asks the linked marketplace to invalidate the old listing. If that callback fails, the transfer reverts.

2. Every explicit Solidity function: Hidden Canvas and artwork

`HiddenCanvasNFT.sol` is the final encrypted ERC-721 collection. Its `Secret` record holds one token's controller, encrypted handle, eligibility block, ownership epoch, publication stage and public value only after verified completion. Sepolia requires exactly **96 blocks** of maturity, both after mint and after a real ownership change. A block count is not a promised number of minutes.

Function	What it does
`constructor(creator, mintPrice, activationDelayBlocks)`	Allows Sepolia or local Hardhat only, checks a real creator and positive price, and enforces 96 blocks on Sepolia. Saves immutable setup values.
`setMarketplace(candidate)`	Creator-only, one-time link to an approved marketplace so transfers can invalidate listings.
`mint()`	Public payable mint with exact fee and 20-token cap. Increments supply and mint proceeds, deploys a fresh per-token controller, calls `FHE.randEuint64()` for an encrypted random seed, permits only the collection and that controller to access its handle, sets epoch 1 and a 96-block eligibility point, and mints the ERC-721 to the payer. No user or server chooses the seed.
`state(tokenId)`	Returns public state: current owner, controller, ciphertext handle, eligibility block, epoch, publication stage and published value. The handle is not plaintext. Before completion the `revealed` field is not a usable private seed.
`activatePrivatePreview(tokenId, expectedEpoch)`	Only the current owner, at or after eligibility, in the sealed stage, can tell the token's controller to grant delegated private-decryption access. The epoch blocks a stale request after a transfer.
`authorizePublicReveal(tokenId, expectedEpoch)`	Only the eligible current owner can call this once, and only while the NFT is unlisted and sealed. It revokes private delegation, sets `PUBLICATION_AUTHORIZED`, and calls `FHE.makePubliclyDecryptable`. **The seed becomes public at this transaction**, even if the browser closes before proof completion.
`completeReveal(tokenId, clearSeed, proof)`	Callable by **anyone** after authorization. It takes the token's *stored* encrypted handle, verifies the Zama proof against that handle and the encoded `uint64` value, then stores the verified public value and marks the token `PUBLISHED`. A bad proof/value or repeated completion reverts. The caller cannot substitute another token's handle.
`canList(tokenId)`	Allows listings only while `SEALED` or `PUBLISHED`; the proof-pending middle stage cannot be listed.

Function	What it does
`artworkHash(tokenId)`	Available only once published. Returns a hash of the fixed renderer version and verified public seed, so a licence remains bound to the same painting after resale.
`tokenURI(tokenId)`	Before completion returns generic metadata and the sealed placeholder, with no private painting or traits. After completion generates an on-chain base64 SVG and public rarity/traits/seed. During proof-pending state it still shows the placeholder, but the seed is **already publicly decryptable**.
`withdrawMintProceeds()`	Only the fixed creator can withdraw accumulated mint fees. It zeroes accounting before sending ETH and uses a reentrancy guard.
`_eligible(tokenId, secret, expectedEpoch)`	Private common check for actual current owner, current ownership epoch and maturity block. An approved transfer operator is not treated as owner.
`_update(to, tokenId, auth)`	Internal ERC-721 ownership hook. On a real owner change it revokes active private delegation if sealed, increments the epoch and restarts the 96-block wait. Any post-mint transfer invalidates a listing, including self-transfer. A self-transfer leaves epoch, maturity and delegation unchanged.

`CanvasArt.sol` is a deterministic renderer library. The frontend counterpart is `art.ts`. The same `uint64` must give the same SVG in both places.

Function	What it does
`rarity(seed)`	Reads the highest byte and selects Common, Rare, Exceptional or Singular by fixed ranges.
`paletteName(seed)`	Chooses one of eight public palette names from the next byte.
`motionName(seed)`	Chooses one of four shape styles from a later byte.
`motifCount(seed)`	Chooses 8–16 shapes.
`palette(seed)`	Returns the four exact colors for the selected palette.
`renderSvg(seed)`	Builds the canonical 640×640 SVG from traits and repeated shape geometry.
`_geometry(state)`	Private helper deriving a motif's position, size and tilt.
`_next(value)`	Private deterministic 64-bit state step; this is **art generation**, not mint randomness. Mint randomness comes from Zama FHE.
`_shape(motion, index, geometry, fill)`	Private helper that renders a rectangle, ellipse, curved path or folded polygon.
`_half(doubled)`	Private helper for exact integer/half coordinates in SVG strings.
`_opacity(index)`	Private helper for repeated opacity values.

The renderer keeps large values as Solidity `uint64`/`uint256` or JavaScript `bigint`; it does not convert a 64-bit seed to an unsafe JavaScript `Number`. The ten Sketches SVGs come from `generate-sketches.mjs`.

3. Every explicit Solidity function: marketplace and licensing

`NFTMarketplace.sol` accepts only the two collection addresses set in its constructor. Listings are **non-escrow**: the seller retains the NFT until a buyer pays. The owner must first approve the marketplace in the ERC-721 contract; that is why listing can produce two MetaMask transactions.

Function	What it does
`constructor(sketches, hiddenCanvas)`	Requires two distinct deployed ERC-721 collections and fixes them permanently.
`isCollectionAllowed(collection)`	Tells a collection whether it is one of those two allowed addresses.
`isListed(collection, tokenId)`	Returns whether a listing is currently active; Hidden Canvas uses this before authorizing a reveal.
`list(collection, tokenId, price)`	Requires linked allowed collection, positive price, listable token, actual owner and marketplace approval. Saves seller/price/current version; the wallet owner signs this transaction.
`cancel(collection, tokenId)`	Only the listing seller can clear an active listing; increments the version.
`buy(collection, tokenId, expectedVersion)`	Rejects old listing versions, self-purchase, wrong payment, changed ownership/approval and non-listable tokens. Clears the listing, credits seller proceeds, then safely transfers the ERC-721. A malicious receiver callback cannot buy the same listing again.
`invalidateFromCollection(tokenId)`	Only a fixed collection contract may call this. Clears its token's listing and increments the version whenever that NFT transfers directly or through the marketplace.
`withdrawProceeds()`	A seller pulls only their own credited Sepolia ETH. The balance is zeroed before payment; a failed payment reverts and preserves it.
`_validateCollection(collection)`	Private check that the address is one of the fixed collections and that it linked this marketplace.
`_approved(collection, tokenId, owner)`	Private check for token-specific or operator approval of the marketplace.

`ArtworkLicenseRegistry.sol` implements a **project-specific educational licence**, not a Story/DATA integration or copyright registration. It has fixed v1 terms, a fee of `0.0001` Sepolia ETH, one receipt per wallet/artwork, and no receipt transfer function. The creator receives the fee even if someone else owns the NFT.

Function	What it does
`constructor(sketches, hiddenCanvas, licenceFee)`	Fixes the two eligible collections and a positive fee.
`termsHash()`	Returns the hash of the fixed text in TERMS; issued receipts retain this version/hash.

Function	What it does
`licenseCount(licensee)`	Returns how many receipts that wallet has.
`licenseReceiptIdAt(licensee, index)`	Gives an indexed receipt ID for the wallet's receipt list.
`obtainLicence(collection, tokenId)`	Requires an allowed collection, exact fee and no previous receipt for that wallet/artwork. Reads the artwork hash and fixed creator, stores an immutable-content receipt with terms version/hash and issuance time, and credits the creator. Sketches works immediately; sealed Hidden Canvas reverts at `artworkHash` until publication is complete.
`withdrawLicenceProceeds()`	Lets the credited creator withdraw their own licence fees using zero-before-send and reentrancy protection.

`IProjectCollection.sol` contains two interfaces rather than deployed contracts. `IListingInvalidator` declares `invalidateFromCollection`, `isListed`, and `isCollectionAllowed` so collections can talk to the marketplace. `IProjectCollection` declares `canList`, `artworkHash`, `creator`, and `marketplace` so marketplace/licensing code can talk to either collection. An interface declares the expected call shape; the actual behavior is in the contracts above.

OpenZeppelin supplies standard ERC-721 methods such as `ownerOf`, `balanceOf`, `approve`, `getApproved`, `setApprovalForAll`, `isApprovedForAll`, `transferFrom`, and `safeTransferFrom`. Solidity also generates read-only getter functions for `public` values and mappings such as `totalSupply`, `creator`, `listings`, `proceeds`, and `receipts`; those getters expose the stored state but do not add custom business logic. The project-specific overrides and functions are explained above.

4. The separate experiment contract, function by function

`SeedFeasibility.sol` was the early privacy gate. Its seed records resemble Hidden Canvas records, but it has **no ERC-721**, marketplace, payments or licences. It should not be mistaken for the finished application.

Function	What it tested
`constructor(delay)`	Enforces Sepolia/local chain and 96 blocks on Sepolia.
`mint()`	Creates one of at most 20 encrypted random seeds and a separate controller; records owner, epoch and maturity.
`state(id)`	Reads public owner/controller/handle/maturity/epoch/publication information.
`activate(id, expectedEpoch)`	Eligible current owner activates delegated private preview while sealed.
`transfer(id, to)`	Experiment-only ownership transfer: rejects bad recipients, revokes old delegation, increments epoch and restarts maturity. Self-transfer leaves access and maturity intact.
`authorize(id, expectedEpoch)`	Eligible owner revokes private delegation and permanently makes the ciphertext publicly decryptable.

Function	What it tested
`finalize(id, clearSeed, proof)`	Anyone with a valid proof can publish the verified value using this seed's stored handle.
`_eligible(seed, expectedEpoch)`	Internal current-owner, epoch and maturity check.

Test-only harnesses in `contracts/test/` are **not** the four deployed application contracts:

Test contract / function	Why it exists
`CanvasArtHarness.renderSvg(seed)`	Exposes the internal Solidity renderer to a local test so its exact SVG can be compared with frontend output.
`RejectEtherSeller.onERC721Received`, `link`, `list`, `withdraw`, `receive`	Acts as a seller contract that can hold/list an NFT but rejects incoming ETH. Tests that failed pull-withdrawal does not lose its credited balance.
`ReenteringBuyer.constructor`, `setTarget`, `buy`, `onERC721Received`, `receive`	Acts as a buyer whose ERC-721 receive callback tries to purchase the same target again. Tests that the marketplace's state changes and reentrancy guard block a duplicate purchase.

5. Why the reveal has two buttons, and how recovery works

> State path: **SEALED** -> owner authorizes permanent reveal -> **PUBLICATION_AUTHORIZED** -> anyone submits a valid proof -> **PUBLISHED**. A browser refresh leaves the middle state on-chain.

The two transactions cannot simply happen in reverse: Zama public decryption needs the **first confirmed transaction** to make the encrypted handle publicly decryptable. The first button is the irreversible privacy decision. The browser then requests a public value **and proof** from the SDK. The second button sends them to `completeReveal`, whose on-chain proof check binds them to that token's stored handle. A UI could later guide both steps in one click sequence, but the blockchain still requires the two ordered stages and a confirmed first transaction.

If the browser closes after authorization, the chain still says `PUBLICATION_AUTHORIZED`. Reopen the NFT, connect a Sepolia wallet and click the **existing "Complete reveal"** button; there is no separate "Recover" button. The contract allows anyone to supply the valid proof, although the website asks for a connected wallet because finalization is a transaction. If proof retrieval or that transaction fails, retry `Complete reveal`; do not click permanent authorization again. Hidden Canvas #2 passed this exact refresh-and-resume test: authorization, then later completion.

Do not call the middle stage private. The public metadata still displays a placeholder while proof is pending, but the seed is already publicly decryptable from the first transaction. Published art and rarity appear only after the valid proof is checked and the final value stored.

6. Frontend and file map

File / function group	Job
`frontend/src/market/chain.ts`: `checkedManifest`, `checkedDraft`	Validates public deployment information, including Sepolia chain and 96-block delay; rejects wrong/malformed deployments.

File / function group	Job
`readProvider`, `walletProvider`, `connectedWallet`, `contracts`	Builds a public read-only Sepolia provider, checks MetaMask's chain, requests the wallet account and creates ethers contract instances from compiled ABIs.
`confirmed`	Waits for a transaction receipt and requires success status; a transaction hash alone is not treated as success.
`deployMarket` and its `validateReceipt`, `deployOne`, `linkOne` helpers	Runs the six MetaMask deployment/link transactions, stores safe public progress for refresh recovery, checks receipts and exact creation data, and returns a manifest.
`frontend/src/market/model.ts`: `readSketch`, `readHidden`, `readListing`, `readMarket`	Reads and verifies the actual Sepolia items, state, listings, licence receipts, fees and balances. A sealed token gets a placeholder, not a private image.
`readInGroups`, `amount`, `short`	Limits batched RPC requests, formats Sepolia ETH, and shortens displayed addresses.
`frontend/src/market/main.tsx`: `savedManifest`, `savedDraft`, `download`, `marketError`	Loads/downloads only public deployment data and maps errors to safer user-facing messages.
`App`, `refresh`, `run`, `connect`, `send`, `withSigner`	Manages navigation and wallet state, refreshes Sepolia state, serializes actions, asks MetaMask to sign, checks receipts, and clears private preview when context changes.
`importManifest`, `deploy`	Reads a manifest from file or deploys via MetaMask, then verifies on-chain state and saves only public data.
`freshPrivatePreview`, `probeNonOwnerAccess`	Uses a fresh signed SDK request for a current owner's private painting; the negative probe counts outbound requests and discards any unexpected plaintext. Ownership is checked again after decryption.
`completeReveal`, `list`, `buy`, `detail`	Obtains the public proof and finalizes reveal; performs explicit approval/listing, checks listing version before purchase, and renders item actions.
`frontend/src/lib/zama.ts`: `initializeBrowserSdk`, `freshInstance`, `freshPreview`, `classify`	Loads Zama WASM, creates a fresh SDK instance, signs a delegated private-decryption permit through MetaMask, and labels explicit ACL rejection separately from an inconclusive timeout. The permit is not an on-chain delegation.
`frontend/src/lib/art.ts`: `traits`, `renderSvg`, `imageUrl`	Converts a verified/private `bigint` seed into the same deterministic traits and SVG as the Solidity renderer.
`scripts/generate-sketches.mjs`	Reproducibly generates the ten SVGs, metadata files, catalog and exact-byte hashes.
`scripts/sync-market-frontend.mjs`, `scripts/sync-zama-assets.mjs`	Copies compiled marketplace ABIs/bytecode, the verified public manifest, catalog and local SDK assets into Vite before dev/build.
`scripts/import-market-manifest.mjs`	Imports downloaded **public** marketplace deployment JSON after checking Sepolia receipts, code, constructor data and configured state. It does not sign transactions.

The app polls public state and clears any private preview if the account, chain, token, owner, epoch or publication context changes. Private seed values stay in page memory. A previous holder can still

remember or save art already seen—the contract cannot erase human knowledge.

7. Why no private key, custom RPC URL or `.env` file is needed

Every deployment, mint, listing, purchase, licence and withdrawal was submitted through **MetaMask**. MetaMask signs internally and gives the website a public account/provider interface. Exporting a key into a project file is unnecessary. No `.env` file is needed for local viewing or wallet-driven transactions.

Optional process variable	Current role	Why it is unnecessary for the website
`SEPOLIA_RPC_URL`	Read by `hardhat.config.ts` **only if a future Hardhat command uses its Sepolia network**. It has a public PublicNode fallback.	Browser deployment used MetaMask; marketplace read-only calls use `PUBLIC_RPC` in `frontend/src/market/chain.ts`. No private RPC credential was needed.
`DEPLOYER_PRIVATE_KEY`	Optional Hardhat CLI signer if explicitly supplied to the process.	MetaMask handled all signed transactions. **Never paste a wallet key into the frontend.**

`hardhat.config.ts` reads `process.env`, but this repository **does not automatically load a root `.env` file**. A future CLI deployment would need deliberately supplied process environment values or a separately reviewed dotenv loader. The current app has no CLI deployment script requiring them. The creator address came from the MetaMask deployer and is stored on-chain; the 96-block delay is also enforced on-chain. `VITE_*` variables would be included in public browser code, so secrets must never go there. The public RPC URL can change if a provider stops working, but it is not a secret. Zama's relayer/KMS endpoints are configured by the installed Sepolia SDK configuration.

The public address/receipt manifest is safe to share. A MetaMask **signature**, private key, wallet seed phrase and an unrevealed artwork seed should not be pasted into chat or documentation. Once an artwork has been publicly authorized, its seed is public by design.

8. CryptX: the finished dark-mode website redesign

The marketplace website is now **CryptX**, while the deployed contract names and `canvas-market` manifest identifier remain as they were on Sepolia. It uses a dark NFT-gallery visual direction inspired by marketplace browsing, **without copying OpenSea's logo, images or exact interface**. The original C/X mark is `cryptx-mark.svg`; the theme and responsive layout are in `style.css`.

What changed:

1. Near-black/navy background, lighter cards and panels, high-contrast headings, muted secondary text and blue action buttons with visible hover/focus states.
2. A compact header with the new wordmark and logo, Explore/Sketches/Hidden Canvas/My NFTs/Licences navigation, searchable artwork names/collections, Sepolia indicator and MetaMask button.
3. A smaller featured collection hero, collection shortcuts, image-led NFT cards, a listed-only filter, and clear fixed-price versus status labels. Sealed tokens still use their public placeholder; private art never appears in a public card.
4. A two-column desktop NFT detail view with artwork, price/owner actions, publication state, traits and licence terms. The public reveal warning and recoverable `Complete reveal` action retain their behavior.

5. Responsive mobile navigation and two-column cards, with no document overflow in the checked 390px layout. The browser still clears private previews on wallet, chain and ownership changes.
6. The frontend build and browser tests passed; live public Sepolia browsing showed 12 artworks, including two published Hidden Canvas tokens. Search returned the two Hidden Canvas items; the listed-only filter correctly returned none for them. No new deployment or private key was needed.

9. Remaining limits

- The contracts are unaudited. Local mocks and browser tests do not prove live confidentiality; the Sepolia demonstration exercised the deployed contracts separately. A relayer timeout is inconclusive, while an explicit ACL label can support a denial finding.
- Delegation can be revoked for future decryptions, but an earlier owner can keep any plaintext or image already seen.
- The pinned Zama SDK and relayer may change. Upgrades require a compatibility and live-access check.
- There is no public hosting or stable metadata domain. Third-party marketplace compatibility, mainnet deployment and legal copyright protection have not been established. The licence receipt records demo terms; it cannot physically prevent copying.

Part III — Run and operate CryptX yourself

1. Your normal start procedure on this Mac

Open **Terminal**. The current workspace path is `/Users/sajan/Desktop/BlockChain`; if you move it, replace that path with its new location. The following commands work even if the server is currently stopped:

```
cd /Users/sajan/Desktop/BlockChain
nvm use 22.23.3
npm --prefix frontend run dev
```

Then open `http://127.0.0.1:5173/` in a browser. Keep Terminal open. The `dev` script runs the two sync scripts, then Vite. Because dependencies and compiled artifacts already exist on this Mac, the short restart path is enough. If either is missing, use the full first-time procedure below.

Do not start a second copy while port 5173 is already occupied. Vite is intentionally set to fail rather than switch to 5174: local NFT metadata expects 5173.

2. Full first-time procedure after a Git clone or on a new Mac

Install `nvm` following its official instructions if it is not present; open a new Terminal after installation. Clone your own repository, or copy the full source folder. Then:

```
cd /path/to/BlockChain
nvm install 22.23.3
nvm use 22.23.3
npm ci
npm --prefix frontend ci
npm run compile
npm --prefix frontend run dev
```

The two `npm ci` commands install two separate lockfiles. `npm run compile` generates the contract artifacts that frontend sync reads. The last command syncs ABI/manifest/Zama assets and starts the website. You need internet for first-time package download, Sepolia reads and private-decryption service access. On Windows, use a Windows-compatible Node 22.23.3 installer/manager and PowerShell; Unix `nvm` commands are not a PowerShell procedure.

Keep `deployments/sepolia-marketplace.json`, both package lockfiles, all source and `frontend/public/art/` plus `frontend/public/metadata/` in the Git repository. Do not commit `.env`, seed phrases or wallet export files. `node_modules/`, `artifacts/`, `frontend/dist/` and `frontend/public/vendor/` are generated/ignored and rebuilt by the commands above.

On a new browser, the bundled manifest selects the **same existing Sepolia contracts**. There is no need to press “Deploy project with MetaMask,” mint ten Sketches again or import a private key into the project. Install MetaMask separately only if you want signed actions; it manages accounts outside the source folder. To act as an already-existing owner, access that same account through MetaMask's supported recovery/migration process. A new wallet may browse and buy but cannot act as another owner's wallet.

3. How to stop the currently running site

The simplest method is to return to the Terminal window running `npm --prefix frontend run dev` and press **Ctrl+C**. That stops the local Vite process. Closing that Terminal also normally stops it. Stopping the local site **does not** delete NFTs, reverse transactions or stop Sepolia contracts.

If you cannot find that Terminal, check the process holding port 5173:

```
lsof -nP -iTCP:5173 -sTCP:LISTEN
```

At the time this handbook was prepared, the listener was a `node` process with PID **92589**. A PID can change, so use the number shown by *your fresh* `lsof` output. After confirming it is your CryptX/Vite process, ask it to stop normally:

```
kill -TERM <PID_FROM_LSOF>
lsof -nP -iTCP:5173 -sTCP:LISTEN
```

The second `lsof` should show no listener. For the currently observed PID, the concrete command would be `kill -TERM 92589` **only if `lsof` still shows that PID**. Do not copy a stale PID blindly. Do not use `kill -9` as the normal stop method. To restart later, repeat the short start procedure above. The old page URL will stop responding while Vite is stopped; that is expected.

4. Everyday website actions and what they cost

Action	What happens	Wallet/gas?
Browse collections or a published NFT	Public read-only Sepolia RPC calls and local assets.	No wallet, no transaction gas.
Connect MetaMask	Allows the page to know the selected address and ask for approvals.	Wallet permission prompt; no on-chain gas by itself.
Mint Hidden Canvas	`mint()` creates an ERC-721, encrypted random seed and controller; pays `0.001` Sepolia ETH plus gas.	One on-chain transaction.
Activate private preview	Eligible current owner activates delegation after 96 blocks.	One on-chain transaction and gas.
Preview privately	MetaMask signs a typed request and the Zama relayer decrypts for the eligible current owner.	Off-chain signature/request; ordinarily no on-chain transaction gas for the preview itself.
List an NFT	Owner approves marketplace, then records a fixed-price listing.	Often two transactions: ERC-721 approval and list.
Buy an NFT	Buyer pays the displayed exact listing price plus gas; owner changes, seller gains withdrawable credit.	One transaction.
Authorize permanent reveal	Current eligible owner makes the seed publicly decryptable. It is irreversible.	One transaction.

Action	What happens	Wallet/gas?
Complete reveal	Browser obtains public value/proof, then contract verifies and publishes traits/SVG. Can be retried after refresh.	One transaction; anyone may supply a valid proof.
Obtain artwork licence	Pays `0.0001` Sepolia ETH to receive a non-transferable receipt for public content.	One transaction.
Withdraw proceeds	Seller or creator withdraws separately credited sale, mint or licence ETH.	One transaction per available balance.

MetaMask shows the amount and testnet gas before approval. Never approve an unfamiliar mainnet request for this Sepolia project. Listing prices are chosen by the seller and are independent of the fixed mint/licence fees.

5. Recheck the project after changes

From the root folder, after dependencies are installed:

```
npm test
npm run typecheck
npm --prefix frontend run build
```

To run browser tests once, install Playwright Chromium from the frontend folder and run its tests there:

```
cd frontend
npx playwright install chromium
npm test
```

At preparation time, **14 Hardhat tests**, **7 Playwright browser tests**, root typechecking and the frontend production build passed. Browser tests use mock wallet conditions and do not send real Sepolia transactions. Local FHE tests cannot alone prove live confidentiality; the historical two-wallet Sepolia demo separately exercised the deployed permissions and reveal process. The PDF is documentation, not a substitute for checking the live chain after future changes.

Part IV — Questions and answers

Q1. Do I need to deploy the contracts again after cloning?
No. The addresses and public receipts are in `deployments/sepolia-marketplace.json`. The frontend sync copies that manifest into its bundled data and reads those same Sepolia contracts.

Q2. Where is the main contract address?
There are four application contracts, not one. The marketplace is `0xc6209DBeF278159DAeb383414C34266D65ec1252`; Sketches is `0xE98bEd0943A1fc22296E1bBfafca30660a3b6508`; Hidden Canvas is `0x8a2d41319C8bF12Cc86a10C2499d195a9dF47a84`; licensing is `0x8a465728f97EEc5062Ed57E46B5659Cfbd7d9E13`.

Q3. Will NFT pictures stay the same on another computer?
Yes when the same source and port 5173 are available. Sketches SVG files are in the project and their exact hashes were stored at deployment. Published Hidden Canvas images are derived from verified on-chain seed values and a fixed renderer. Do not alter the original SVGs if you want them to keep matching on-chain hashes.

Q4. Why do some external NFT marketplaces fail to show these pictures?
Sketches token metadata and the sealed Hidden Canvas placeholder refer to `127.0.0.1:5173`, which means the viewer's own computer. A remote marketplace cannot access your localhost. Published Hidden Canvas SVG data is on-chain, but the local-URL issue remains for the other assets. Hosting the current frontend alone cannot rewrite URLs already stored in the deployed contracts.

Q5. Why is there no `.env` file or Sepolia API key?
Public reads use the configured PublicNode Sepolia RPC; MetaMask signs transactions in the wallet extension. A public RPC URL is an endpoint, not a private key. No paid provider account was needed for this demo. Optional Hardhat CLI environment variables exist in `hardhat.config.ts`, but the browser workflow does not need them.

Q6. Do I need to give CryptX my MetaMask private key or recovery phrase?
No. MetaMask manages the account. CryptX requests wallet actions; it never needs an exported key in source, a `.env` file, Git, chat or the PDF.

Q7. Why does MetaMask still ask for Sepolia ETH if the RPC is free?
The RPC transports requests; Sepolia gas pays validators for state-changing transactions. Mint and licence fees are additional project-contract payments. Free RPC access does not make transactions gas-free.

Q8. What is the difference between a wallet seed phrase and an artwork seed?
A wallet seed phrase controls an Ethereum account and must stay private. A Hidden Canvas artwork seed is an FHE-generated `uint64` that determines one painting. It begins encrypted, may be privately learned by an eligible current owner, and becomes intentionally public after reveal authorization.

Q9. Can I preview immediately after minting or buying Hidden Canvas?
No. Sepolia requires **96 blocks** after mint or real ownership change, then the current owner must activate preview. A block count is not a guaranteed time in minutes.

Q10. Why are “Reveal publicly” and “Complete reveal” separate?
The first transaction authorizes public decryption and irreversibly exposes the seed. Only after it confirms can the browser obtain a public decryption proof. The second transaction checks that proof on-chain and publishes the verified SVG/traits. If interrupted, reopen the NFT and retry **Complete reveal**; there is no separate recovery button.

Q11. Can a former owner decrypt again after selling a sealed NFT?
The transfer revokes their active delegation and starts a new epoch/maturity period for the buyer. A fresh former-owner request should be denied. A former owner may still remember or have saved any plaintext seen earlier; the contract cannot erase memory or copies.

Q12. Is an NFT sale the same as selling copyright or the artwork licence?
No. The NFT changes owner. The creator's licensing role stays separate; `ArtworkLicenseRegistry` issues a non-transferable receipt under its fixed demo terms. It is an educational on-chain record, not a legal guarantee against copying.

Q13. Why can Sketches be licensed immediately but sealed Hidden Canvas cannot?
Sketches stores an artwork hash from construction. Hidden Canvas exposes `artworkHash` only after verified public reveal, so sealed or proof-pending artwork cannot be the basis of a public-content licence.

Q14. Why might listing require two MetaMask confirmations?
ERC-721 ownership does not automatically grant the marketplace transfer permission. The owner first approves the marketplace, then signs the `list` transaction. A previous approval can reduce that to one new transaction.

Q15. Why does a sale not immediately send ETH to the seller's wallet?
The marketplace credits `proceeds[seller]` and the seller uses `withdrawProceeds()`. This pull-payment design separates purchase/transfer from payout and makes failure handling safer.

Q16. What happens if I stop the local website?
Only your local Vite web server stops. NFTs, owners, listings, receipt balances and reveals remain on Sepolia. Start Vite again to view them.

Q17. What if port 5173 is busy?
Find the listener with `lsof -nP -iTCP:5173 -sTCP:LISTEN`, stop your own old Vite process with Ctrl+C or `kill -TERM` after confirming its PID, then restart. This project deliberately does not auto-switch to port 5174 because deployed local metadata uses 5173.

Q18. Why does the manifest still say `canvas-market` instead of CryptX?
That is an internal identifier used by the original deployment and validation code. CryptX is the current website brand. Changing a local display name cannot alter already-deployed contract bytecode, NFT descriptions or historical transactions.

Q19. Where did the old lab page and screenshots go?
The standalone experiment page and saved screenshots were removed from the finished frontend. The feasibility Solidity source and tests remain to document the original development/validation path. The current website has one page: `/`.

Q20. Is the project finished and ready for production/mainnet?
The stated Sepolia coursework features and two-wallet demo were completed. This is **not** an audited production marketplace. Public hosting, permanent non-local metadata, third-party marketplace compatibility, security review and legal licence enforcement remain separate work.

Q21. Can I use a different MetaMask account on a new Mac?
Yes for browsing and any action that account is authorized to perform. It cannot act as the old owner. To manage NFTs already owned by Account A or B, access the same wallet account through MetaMask's official secure recovery/migration process; never put recovery material in this project.

Q22. If the page is empty after moving the project, what should I check?
Confirm `deployments/sepolia-marketplace.json` was copied, Node is `22.23.3`, both `npm ci` commands succeeded, Solidity compiled, Vite runs at port 5173, and the browser can reach Sepolia. Restart `npm --`

`prefix frontend run dev` so the sync script recopies the manifest. An empty `My NFTs` view can also mean no wallet is connected or the selected wallet owns none.

Q23. Are automated tests the same as a real Sepolia transaction test?
No. Hardhat/Playwright tests catch local logic and UI errors without using the live wallet. The earlier two-wallet Sepolia run demonstrated actual owner preview, relayer denial, transfer revocation, public proof, trade and licence flows. Future code or SDK changes should be checked again before making new claims.

Q24. Can I safely push this to Git?
The source, lockfiles, public manifest and artwork should be included. `.gitignore` excludes generated dependencies/build outputs and `.env`. Review `git status` before committing so no wallet exports, recovery phrases or private credentials are included. A fresh clone can follow the full first-time procedure above.

Final reference

The concise startup guide is `docs/SETUP_NEW_DEVICE.md`; the source explanation is `docs/PROJECT_WALKTHROUGH.md`; deployed public addresses are in `deployments/sepolia-marketplace.json`. The local site is `http://127.0.0.1:5173/` while Vite is running. The command to **stop** the active Terminal session is **Ctrl+C**.